

FPGA-based Network Switch Architecture Supporting Credit Based Shaper for Time Sensitive Networks

Akram BEN AHMED, Takahiro HIROFUCHI, Takaaki FUKAI

Digital Architecture Research Center

National Institute of Advanced Industrial Sciences and Technology

This work is based on results obtained from
“Research and Development Project of the Enhanced Infrastructures for Post-5G Information and Communication Systems”(JPNP20017),
commissioned by the New Energy and Industrial Technology Development Organization (NEDO)

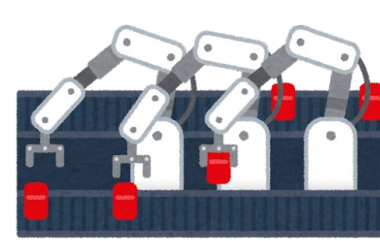
Time Sensitive Network (TSN)

- IEEE 802.1 Time-Sensitive Network (TSN) is a set of standards of network technology

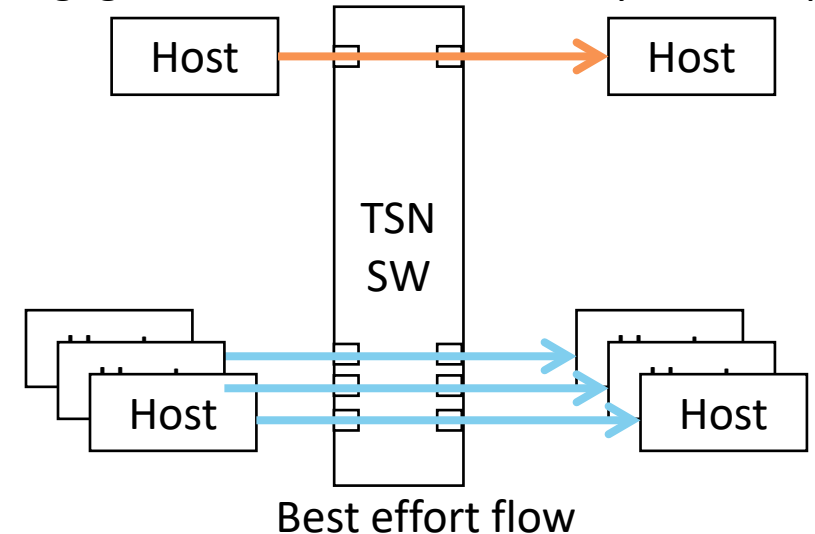
- ✓ Use-case: Factory automation, in-vehicle networks, Avionics network, etc.

- One of important feature of TSN is to guarantee **latency** and **throughput** in Ethernet networks.

- It avoids interference by best effort flows



Time sensitive flow
e.g. guaranteed <100 us latency, 100 Mbps



Motivation

Most of the work evaluate TSN functionalities by simulations or using commercial off-the-shelf hardware

- Hard to evaluation on real (physical) environment with fine-grained analysis
- Lack of unified evaluation hardware platform to compare the methods

Existing works

- ✓ Isolated implementation of TSN functionalities without a complete and comprehensive switch architecture

No comprehensive switch architecture to evaluate TSN methods!

Goal and Contribution

Goal:

- Developing a practical open-source FPGA-based network switch complying with TSN requirements.

Contribution of this work:

- Providing an **FPGA-based hardware** support and **preliminary evaluation** for TSN leveraging Credit Based Shaping (**CBS**) as a traffic shaping algorithm.
- As open source: <https://github.com/CCIRT/aist-tsn> .

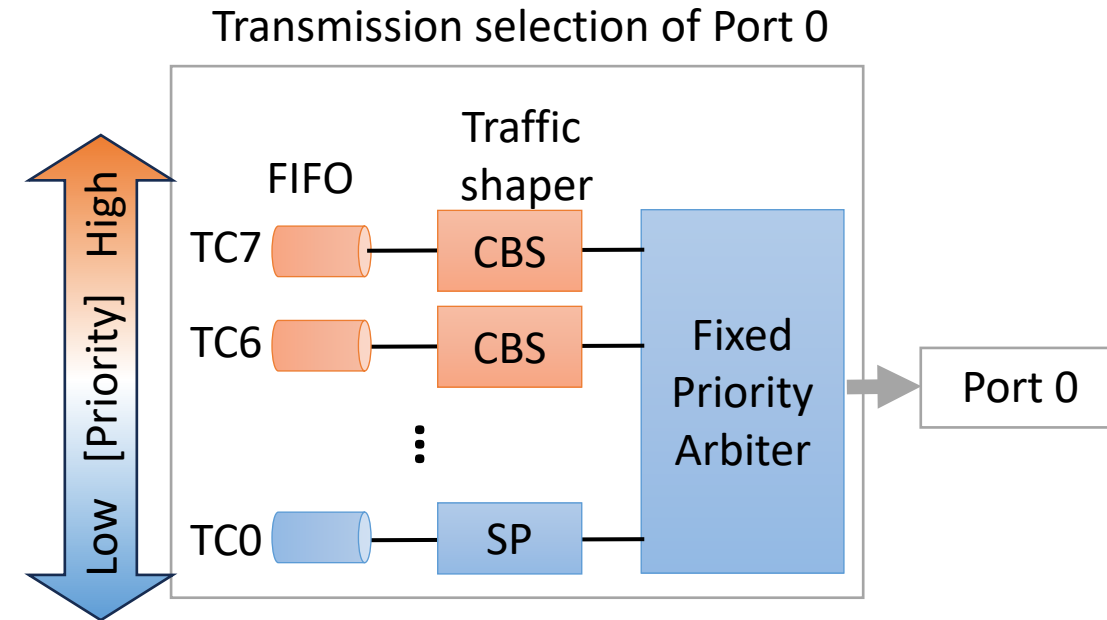


Outline

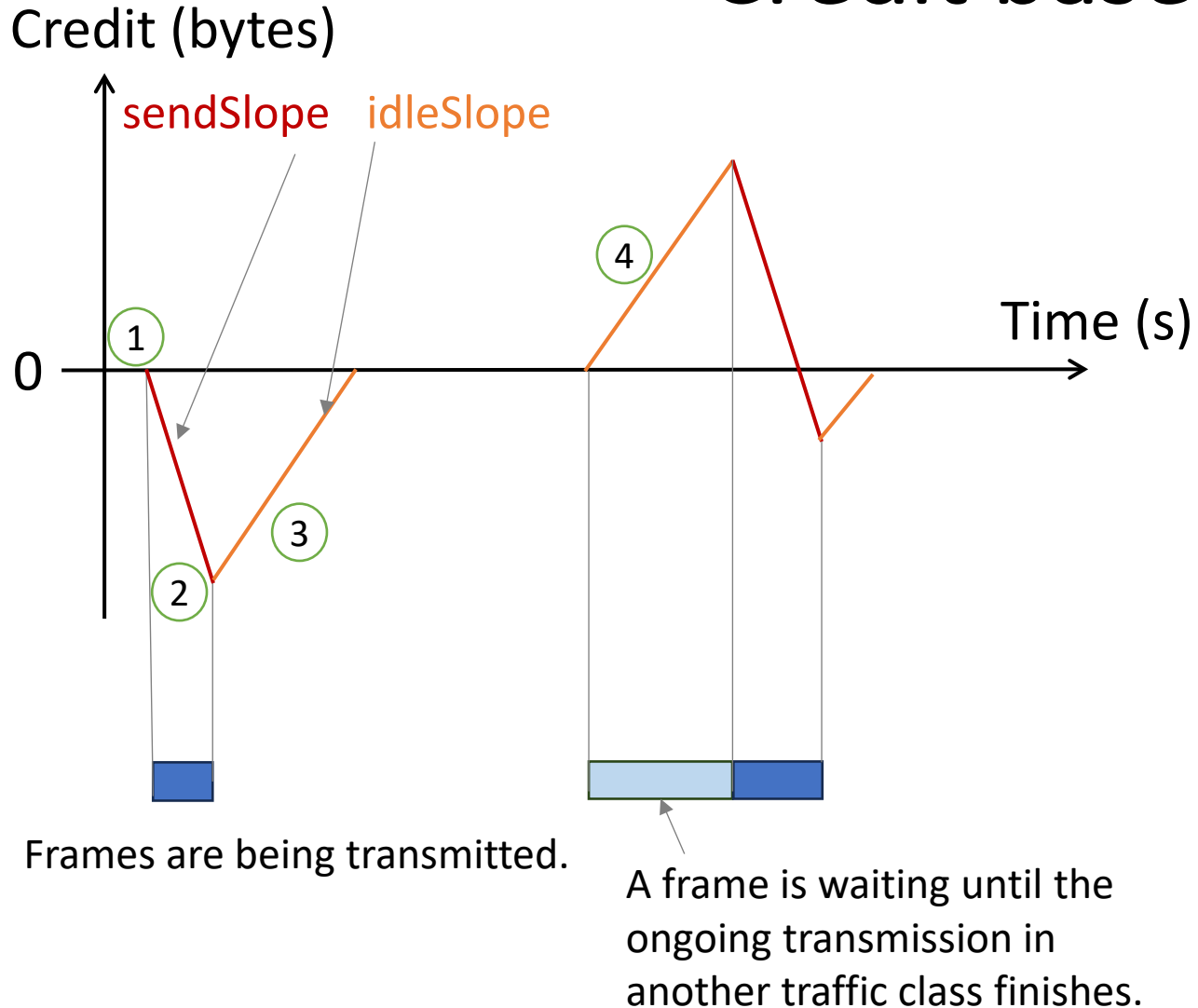
- Technical Background
- Proposed Switch Architecture
- Preliminary Evaluation
- Conclusion

Transmission selection in TSN Switch

- Up to eight traffic classes (TC) per egress port
 - Each frame is assigned to a traffic class based on its QoS priority.
- Traffic shaper: Controlling the timing of the frame transmission for each TC
 - <Examples of traffic shaping algorithm>
 - Strict Priority (SP) = no control
 - Credit Based Shaping (CBS)
 - Asynchronous Traffic Shaping (ATS)
- Fixed Priority Arbiter: Picking a highest priority frame that is ready to be transmit



Credit based shaping

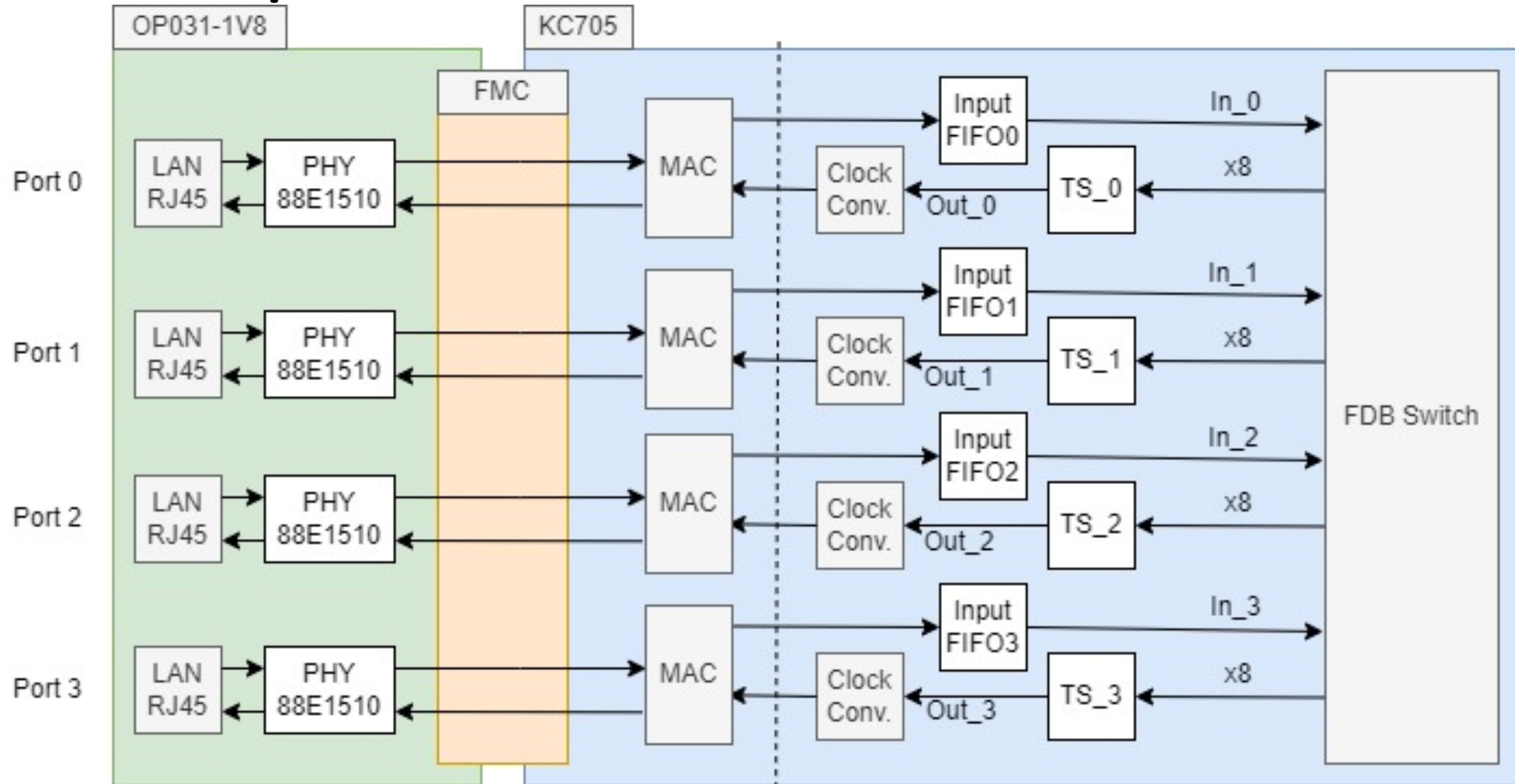


1. A frame can be transmitted when the credit is a zero or positive value.
2. Reducing credit by ***sendSlope*** during frame transmission
3. Increasing the credit by ***idleSlope*** to zero after the frame transmission
4. Increasing the credit by ***idleSlope*** beyond zero during conflict with a transmission in another traffic class.
 - The time to be ready for the next transmit comes earlier for compensation.

Outline

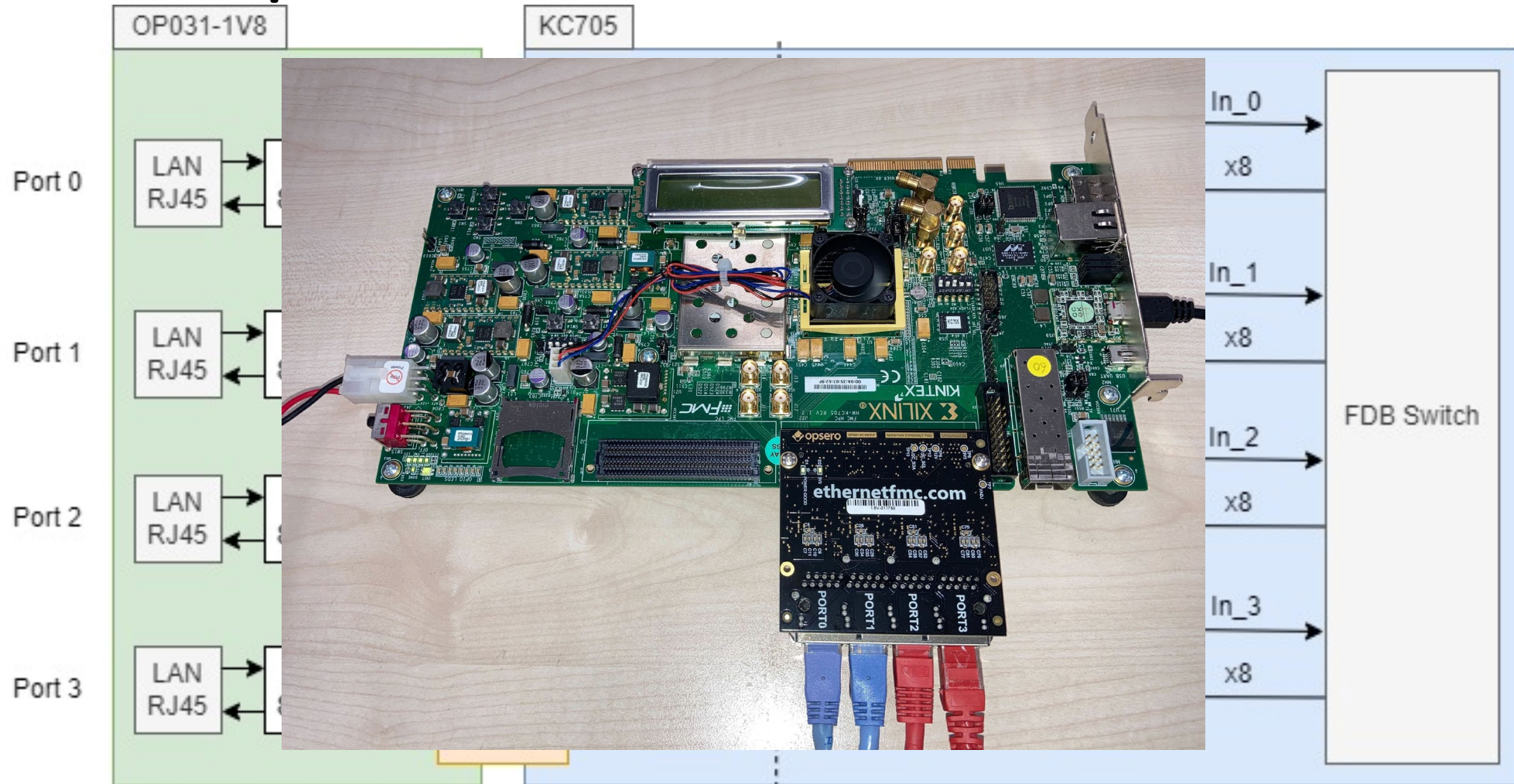
- Technical Background
- **Proposed Switch Architecture**
- Preliminary Evaluation
- Conclusion

Proposed FPGA-based TSN switch



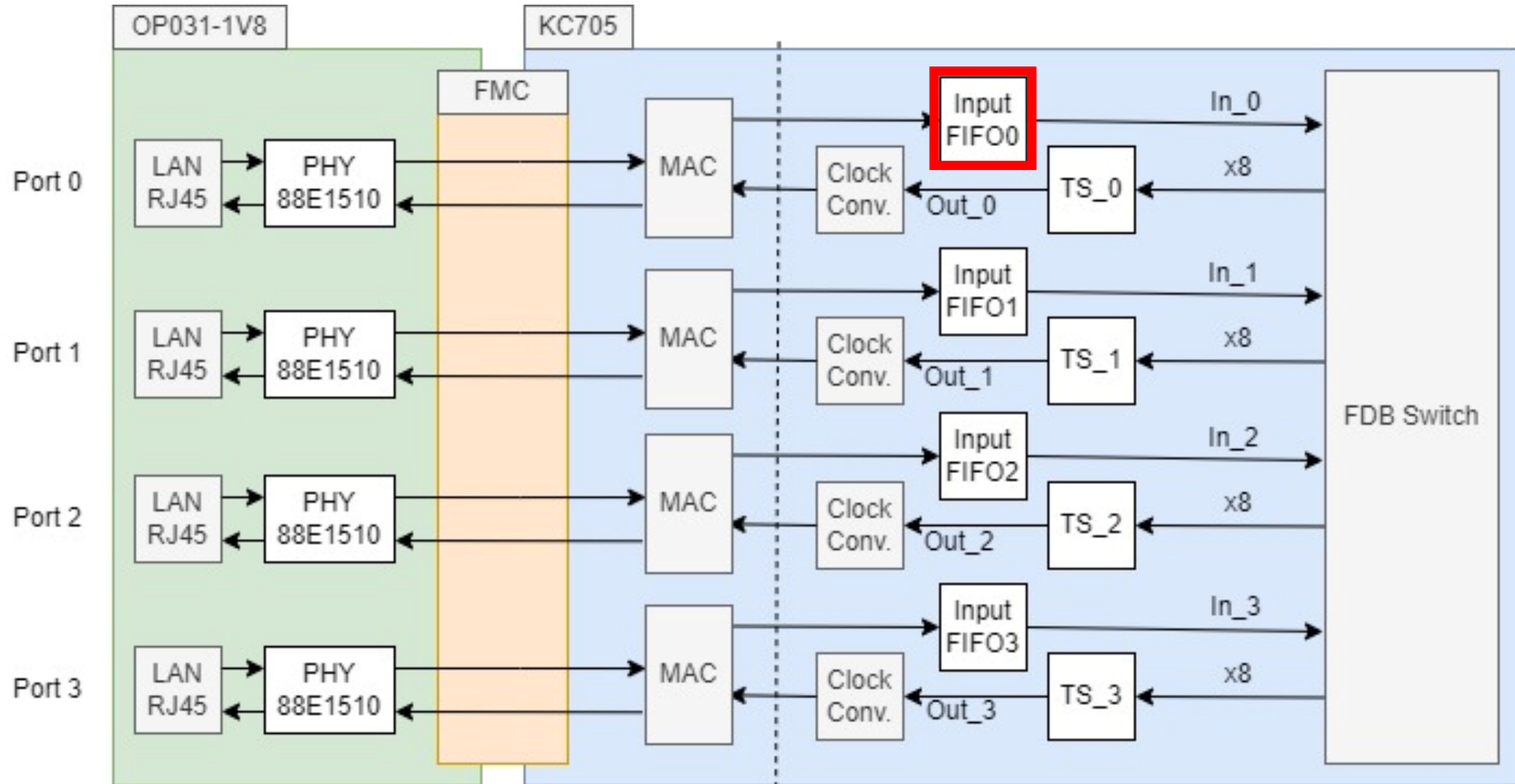
- 1000BASE-T with a maximum designed speed at 1 Gbps and Maximum Transmission Unit (MTU) to be 1500 bytes.
- Ethernet II and IEEE 802.3ac (VLAN tag support)

Proposed FPGA-based TSN switch



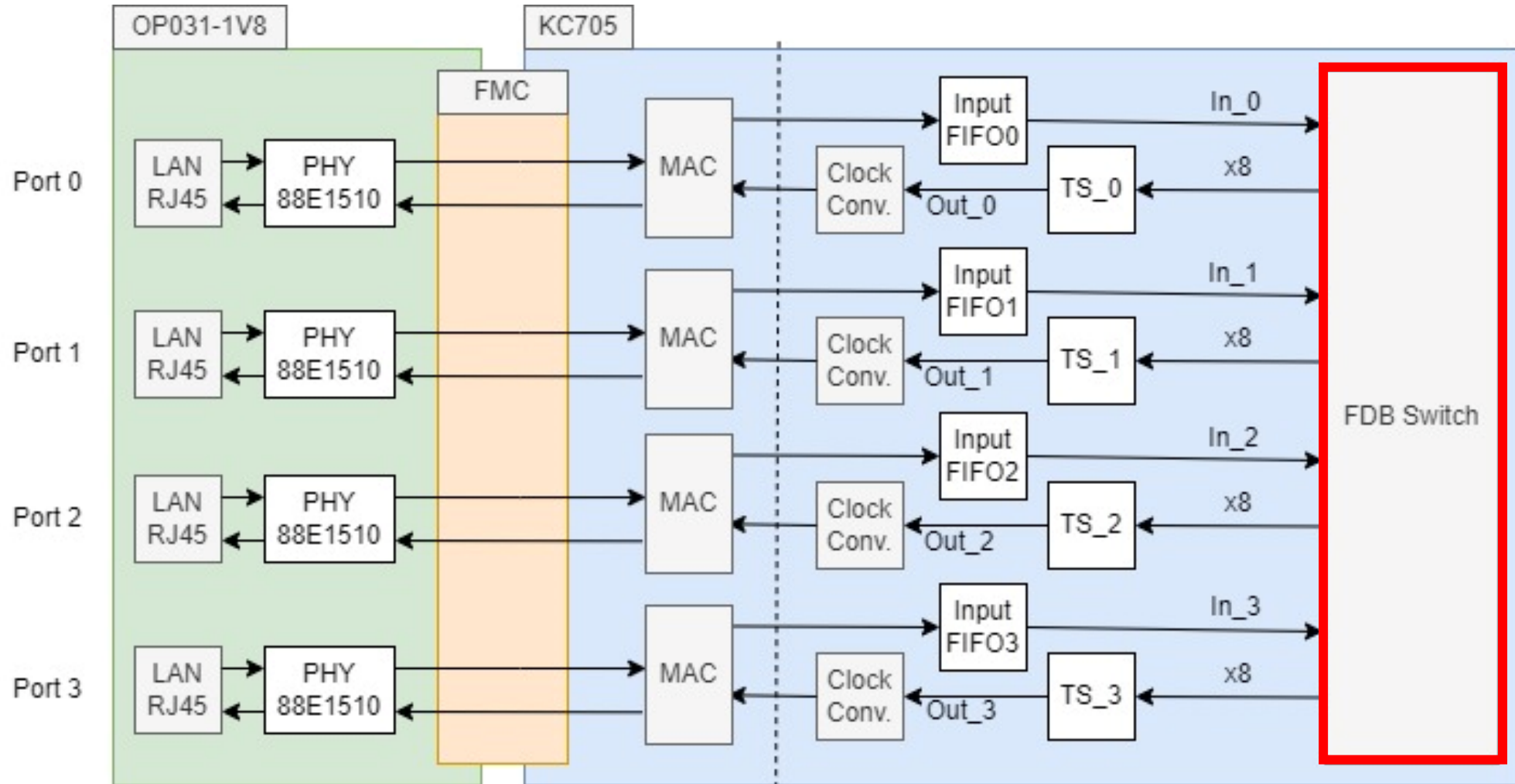
- 1000BASE-T with a maximum designed speed at 1 Gbps and Maximum Transmission Unit (MTU) to be 1500 bytes.
- Ethernet II and IEEE 802.3ac (VLAN tag support)

Pre-switch FIFO block



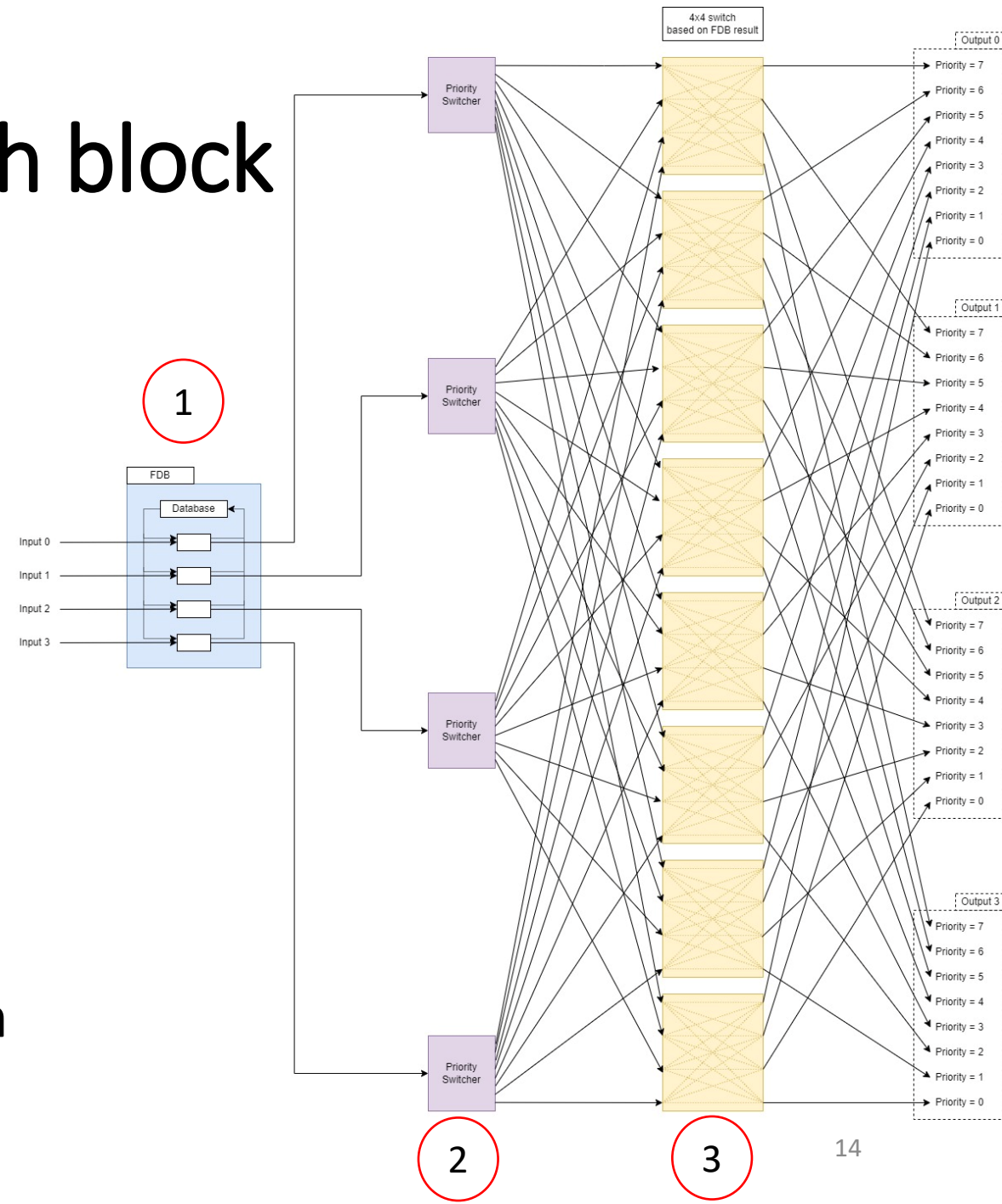
- Deployed for packet storing and for clock synchronization between the clock domain of MAC and that of the FPGA main. Both are at 125MHz, not synchronized.

FDB Switch block



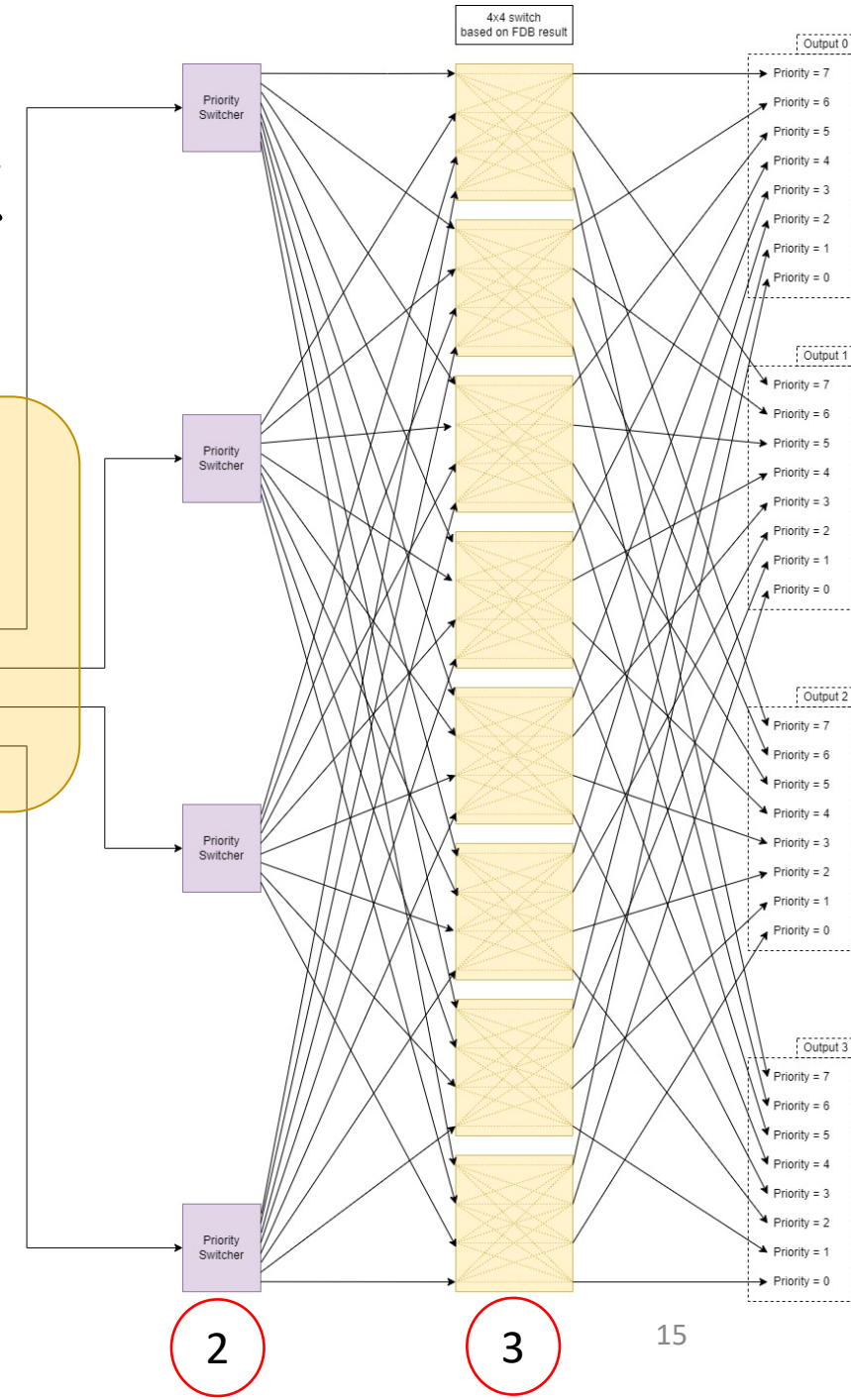
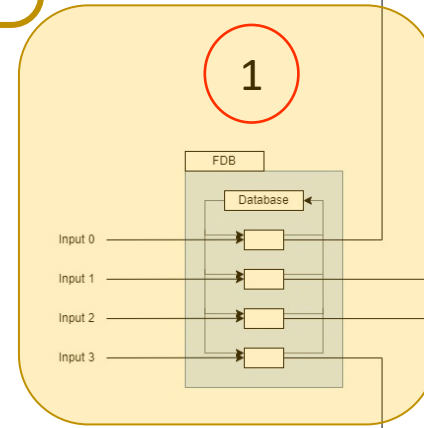
FDB Switch block

1. Determining the egress port based on filter data base (FDB)
2. Switching based on priorities:
 - ✓ from switches per ingress port
 - ✓ to switches per TC
 - ✓ based on priority of frame
 - ✓ indicated in PCP field in frames
3. Switching based on egress port:
 - ✓ from switches per TC
 - ✓ to queues in Transmission Selection
 - ✓ based on the egress port information
 - ✓ (i.e. based on the FDB results)



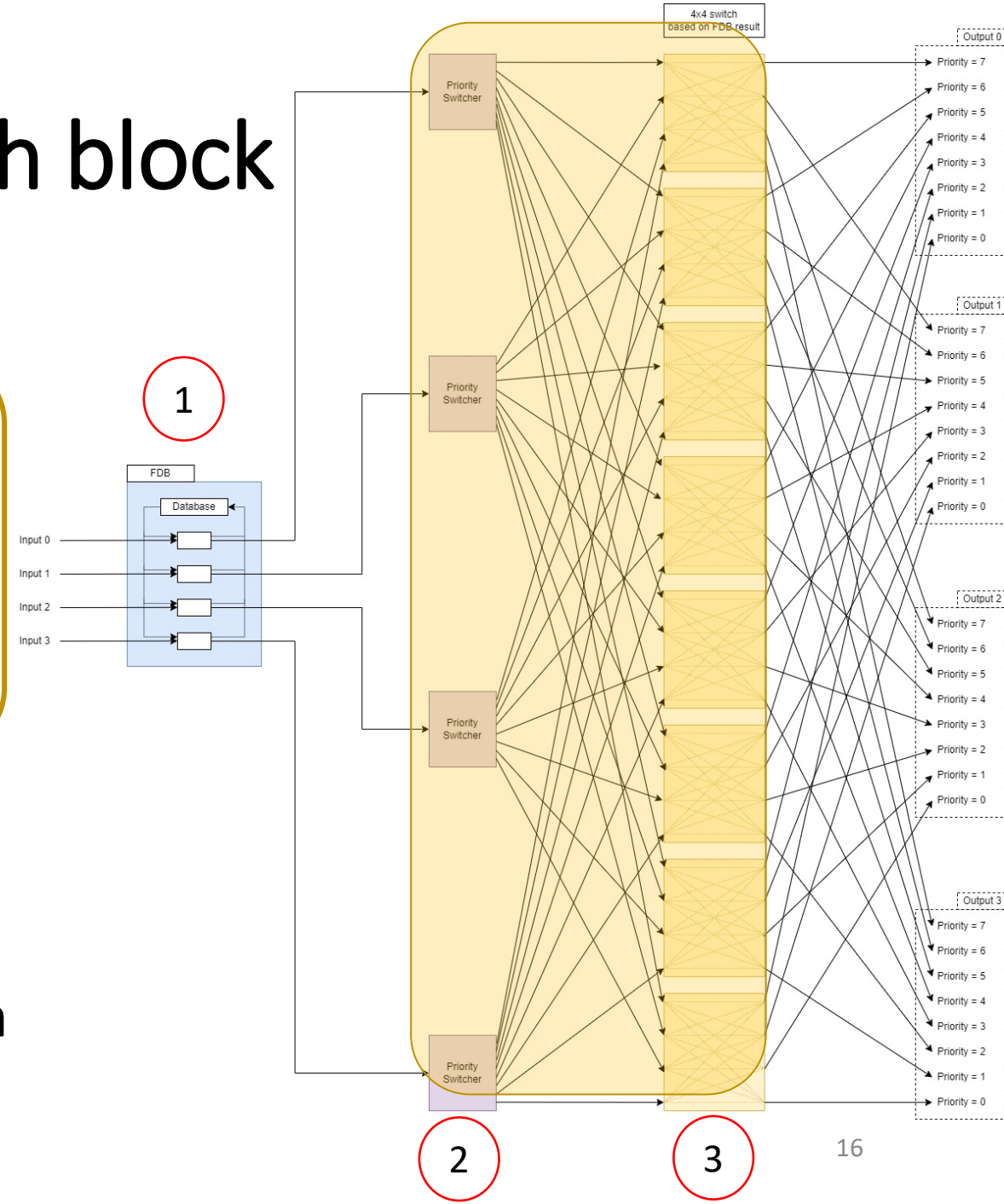
FDB Switch block

1. Determining the egress port based on filter data base (FDB)
2. Switching based on priorities:
 - ✓ from switches per ingress port
 - ✓ to switches per TC
 - ✓ based on priority of frame
 - ✓ indicated in PCP field in frames
3. Switching based on egress port:
 - ✓ from switches per TC
 - ✓ to queues in Transmission Selection
 - ✓ based on the egress port information
 - ✓ (i.e. based on the FDB results)



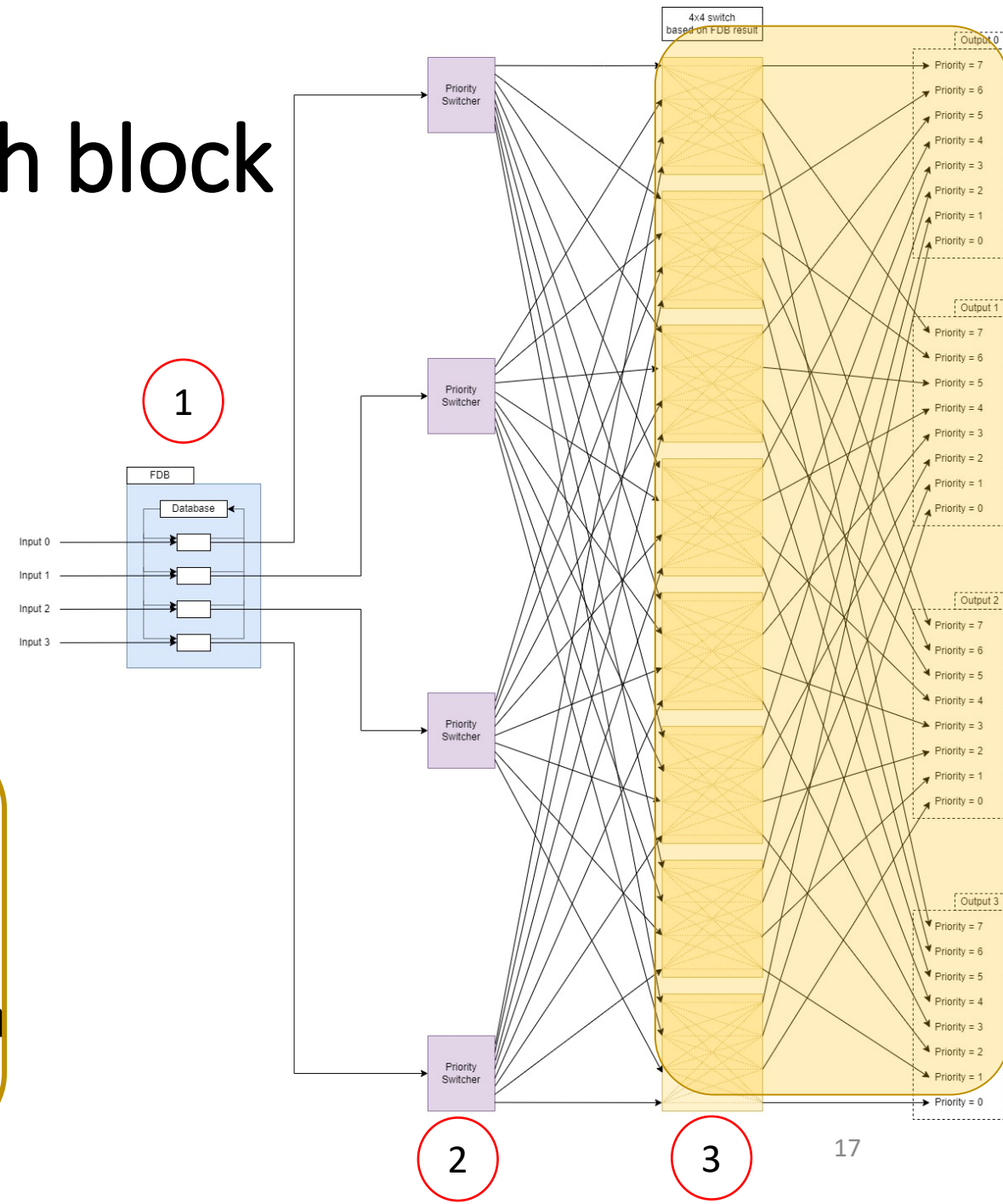
FDB Switch block

1. Determining the egress port based on filter data base (FDB)
2. Switching based on priorities:
 - ✓ from switches per ingress port
 - ✓ to switches per TC
 - ✓ based on priority of frame
 - ✓ indicated in PCP field in frames
3. Switching based on egress port:
 - ✓ from switches per TC
 - ✓ to queues in Transmission Selection
 - ✓ based on the egress port information
 - ✓ (i.e. based on the FDB results)

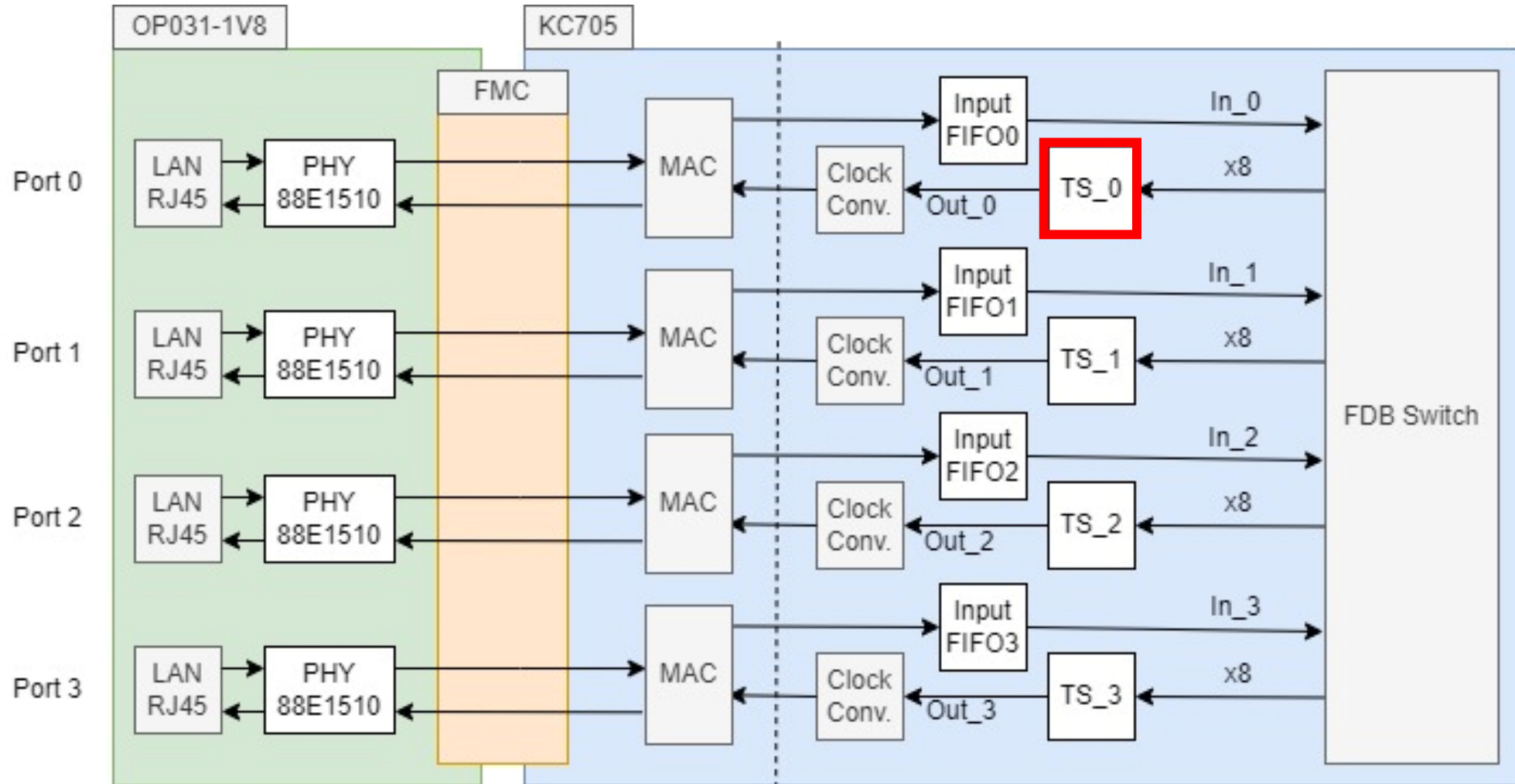


FDB Switch block

1. Determining the egress port based on filter data base (FDB)
2. Switching based on priorities:
 - ✓ from switches per ingress port
 - ✓ to switches per TC
 - ✓ based on priority of frame
 - ✓ indicated in PCP field in frames
3. Switching based on egress port:
 - ✓ from switches per TC
 - ✓ to queues in Transmission Selection
 - ✓ based on the egress port information
 - ✓ (i.e. based on the FDB results)

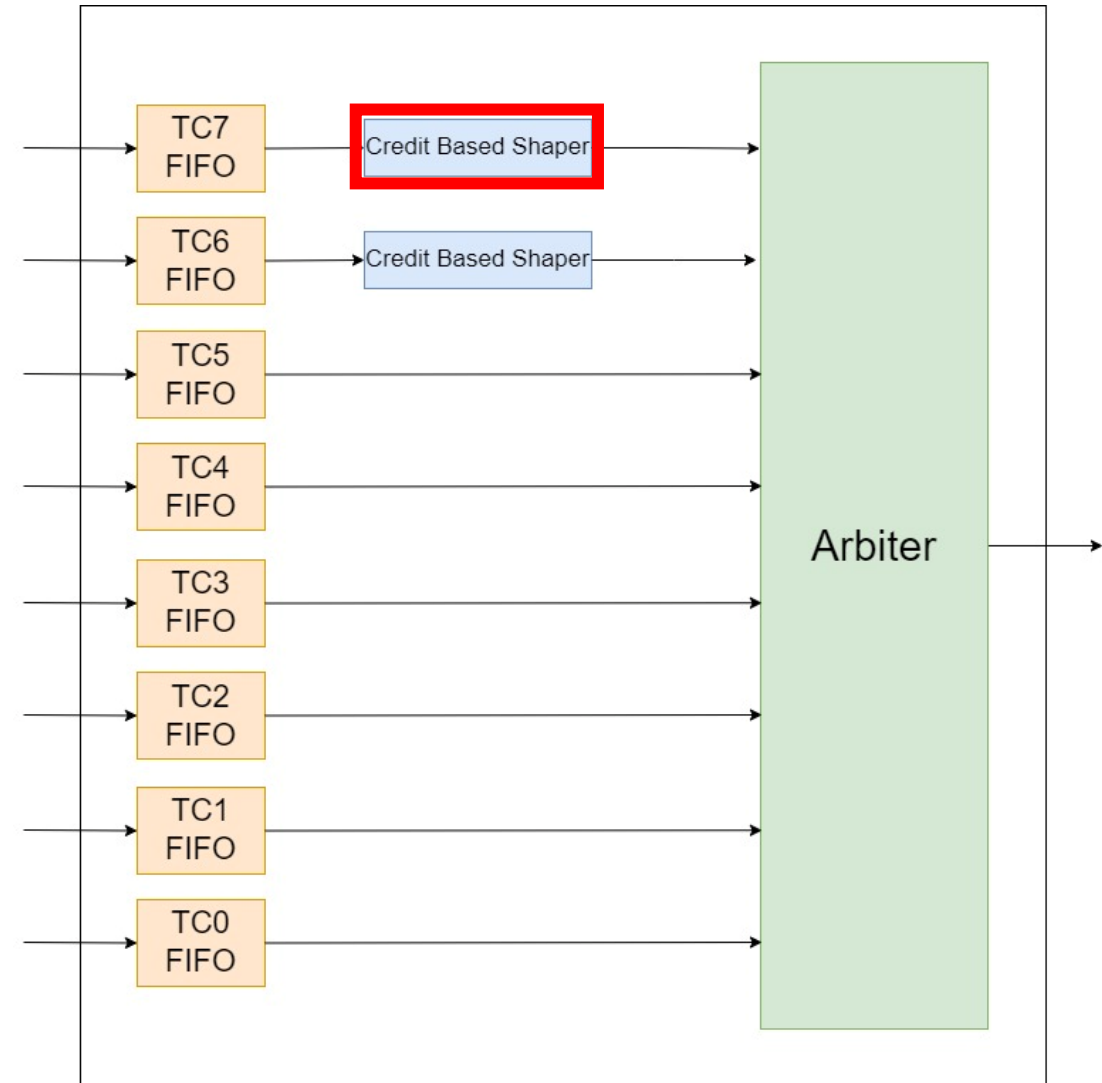


Transmission Selection block



Transmission Selection block

- For Traffic class “7” or “6”, streams are forwarded to the “Credit based shaper” module:
 - ✓ *idleSlope* and *sendSlope* are managed
- Arbiter gives priority to frames with high Traffic Class and outputs them.
 - ✓ Switching is not performed during frame output, but rather earlier in the “FDB Switch”



How the credit is counted in the Verilog source code

```
// Utility wires
wire transmit_allowed = (credit_reg >= 0) | transmit_until_frame_end_reg;
```

If the credit is zero or positive, transmission is allowed.

```
// Credit counter
wire signed [31:0] tmp_idle_credit = credit_reg + idle_slope;
wire signed [31:0] tmp_send_credit = credit_reg + send_slope;
```

In the idle state, the credit is increased

```
always @ (posedge clk) begin
    if (!transmitting) begin
        // idling
        credit_reg <= tmp_idle_credit;
    end else begin
        // sending
        credit_reg <= tmp_send_credit;
    end
end
end
```

In the sending state, the credit is decreased.
send_slope is a negative value.

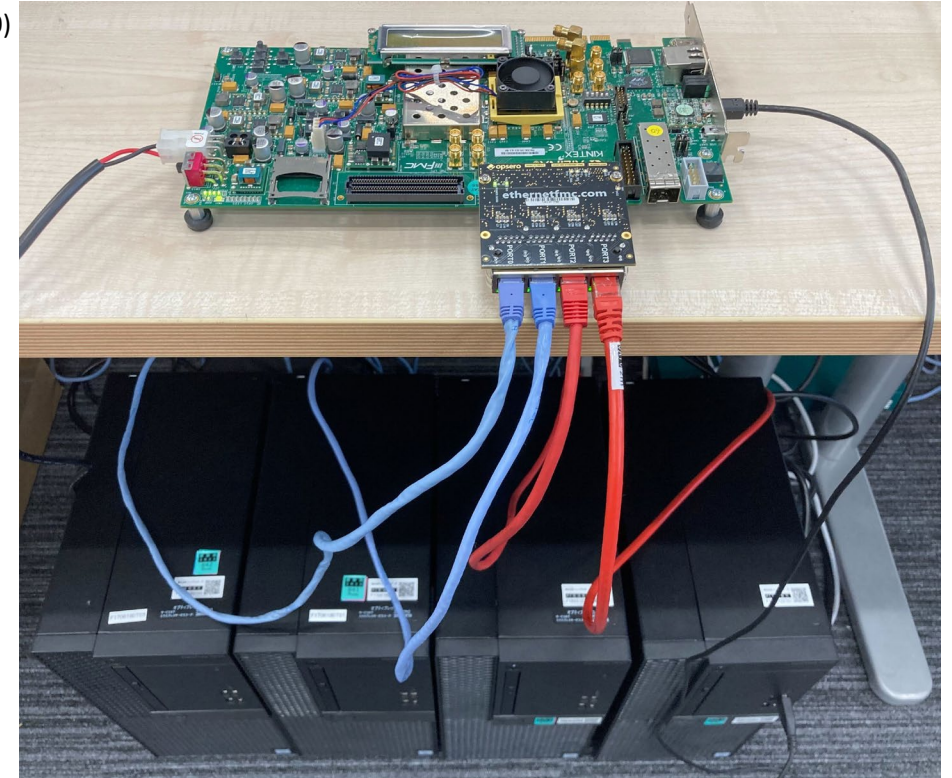
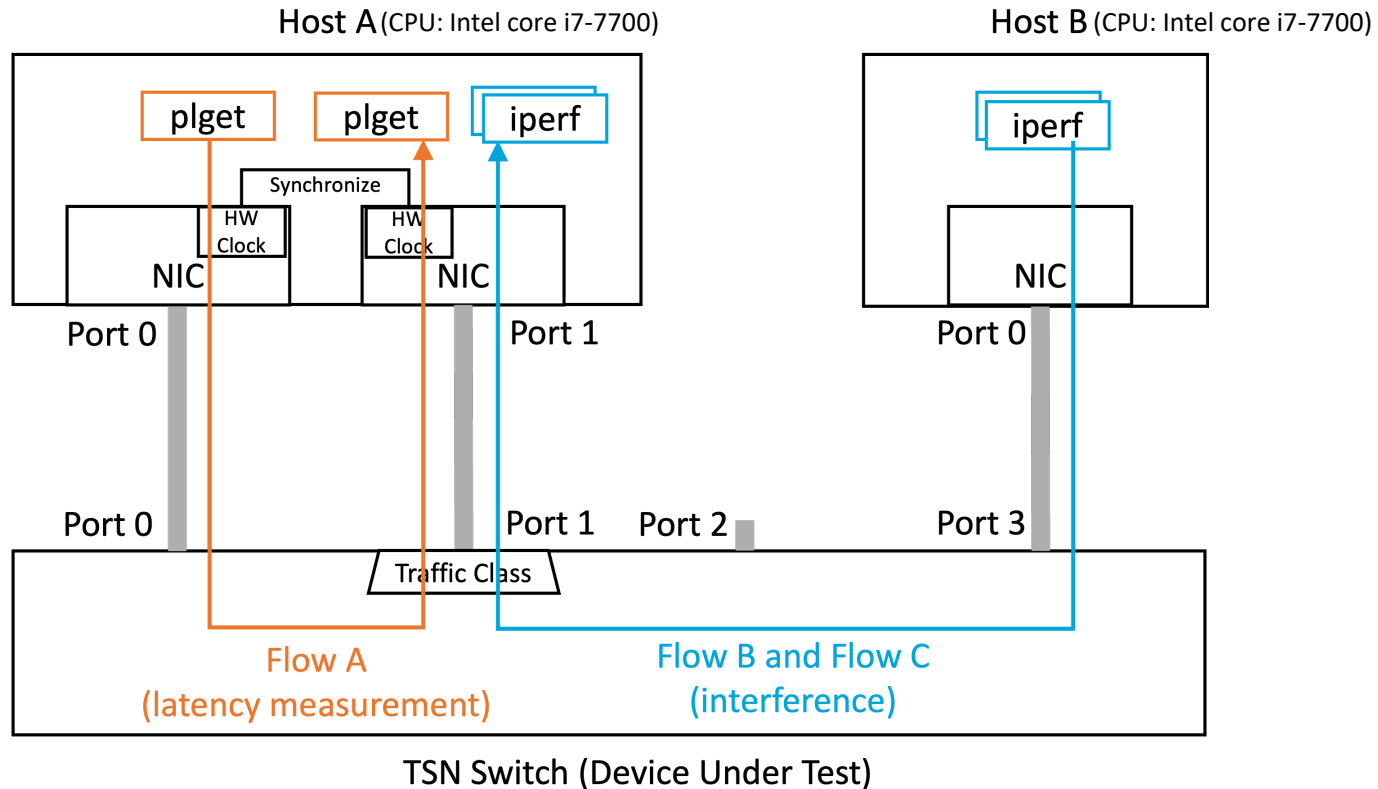
The credit is updated at each positive clock edge

Note that the above code is simplified for ease of understanding.
The original code is located in ./device/rtl/src/cbs/credit_based_shaper.v .

Outline

- Technical Background
- Proposed Switch Architecture
- **Preliminary Evaluation**
- Conclusion

Evaluation Environment



- Host A has two GbE NICs (Intel i210) and Host B has one GbE NIC.
- **iperf** for frame generation and minor customization of **plget** to measure latency
- The payload size of an Ethernet frames is set to 1500 bytes

How is latency measured?

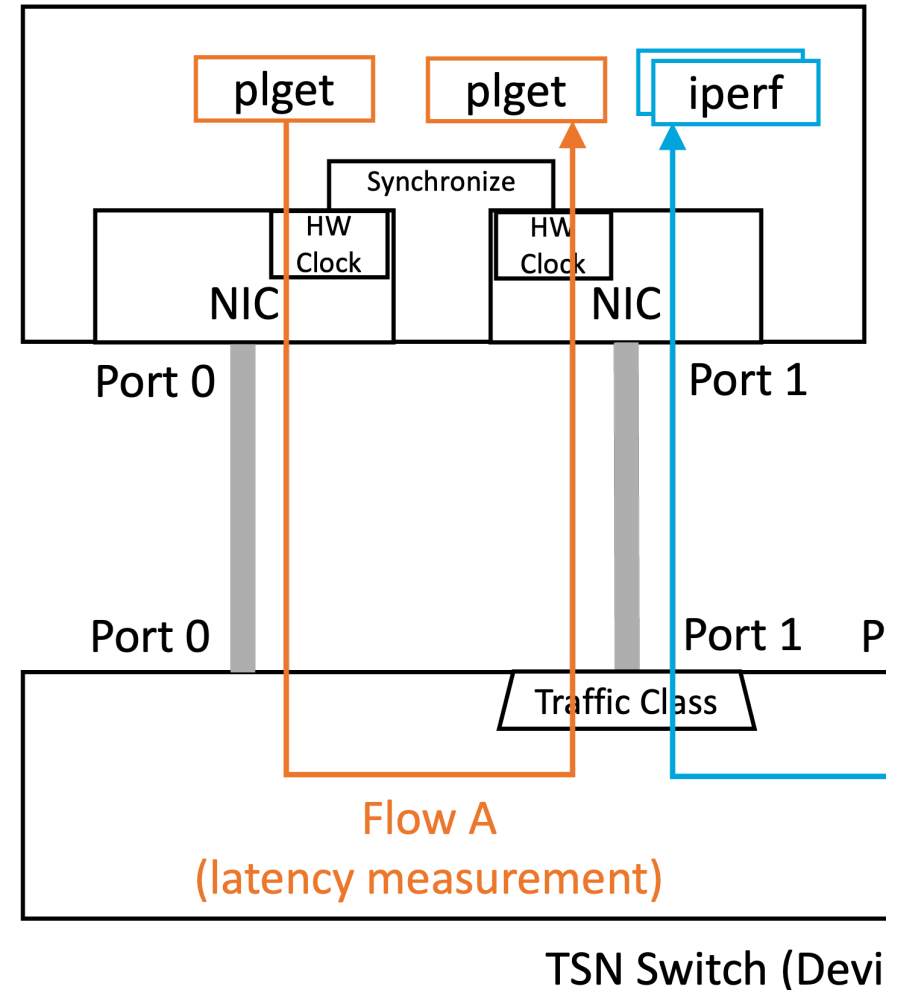
Host A

Setup

- Using hardware timestamp function of Intel i210
- Synchronizing two hardware timestamp clocks by the PTP daemon on Host A periodically.

Precision and limitation

- Due to this synchronization overhead, we discuss latencies in a 1-us resolution.
- Due to the limitation of Intel i210, we transmit frames at 100 frame per second for the flow of latency measurement.

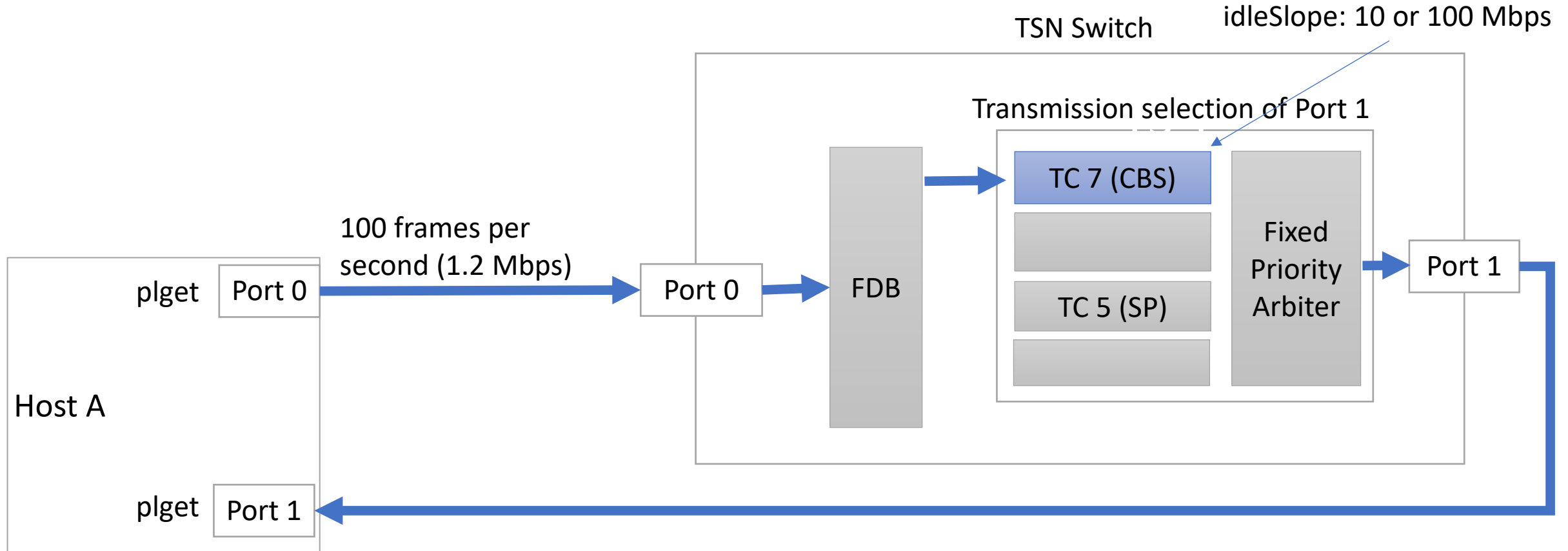


Preliminary experiments

Experiments to verify our implementation step-by-step

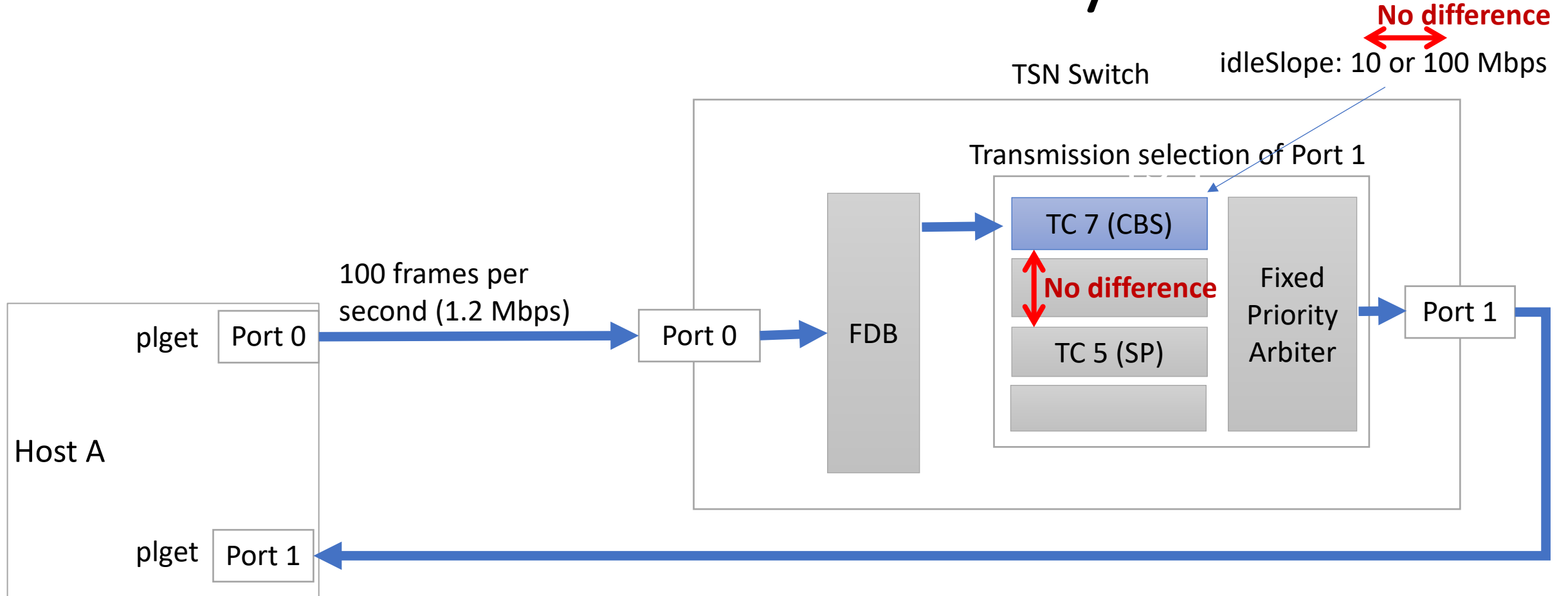
- Network switch basic latency
 - ... How low overhead on the network switch by our architecture
- CBS Traffic Class Rate Control
 - ... If correctly working the rate control by CBS with single flow
- Strict Priority Control
 - ... If correctly working the strict priority control with two flows
- Latency bound by CBS
 - ... If correctly working the rate control by CBS with an interfering flow

Network switch basic latency: Setup



- Measured by the hardware timestamp and Integrated Latency Analyzer in FPGA
- Benchmark: plget
- Flow: Through TC7 (idleSlope=10 or 100Mbps), TC5 (Strict priority)

Network switch basic latency: Result



- The latency was 62 us for a 1500B-payload frame and 4 us for a 64B-payload frame.
- No difference among CBS and Strict Priority was observed.
- No difference among idleSlope values was observed.

Latency breakdown in each FPGA component

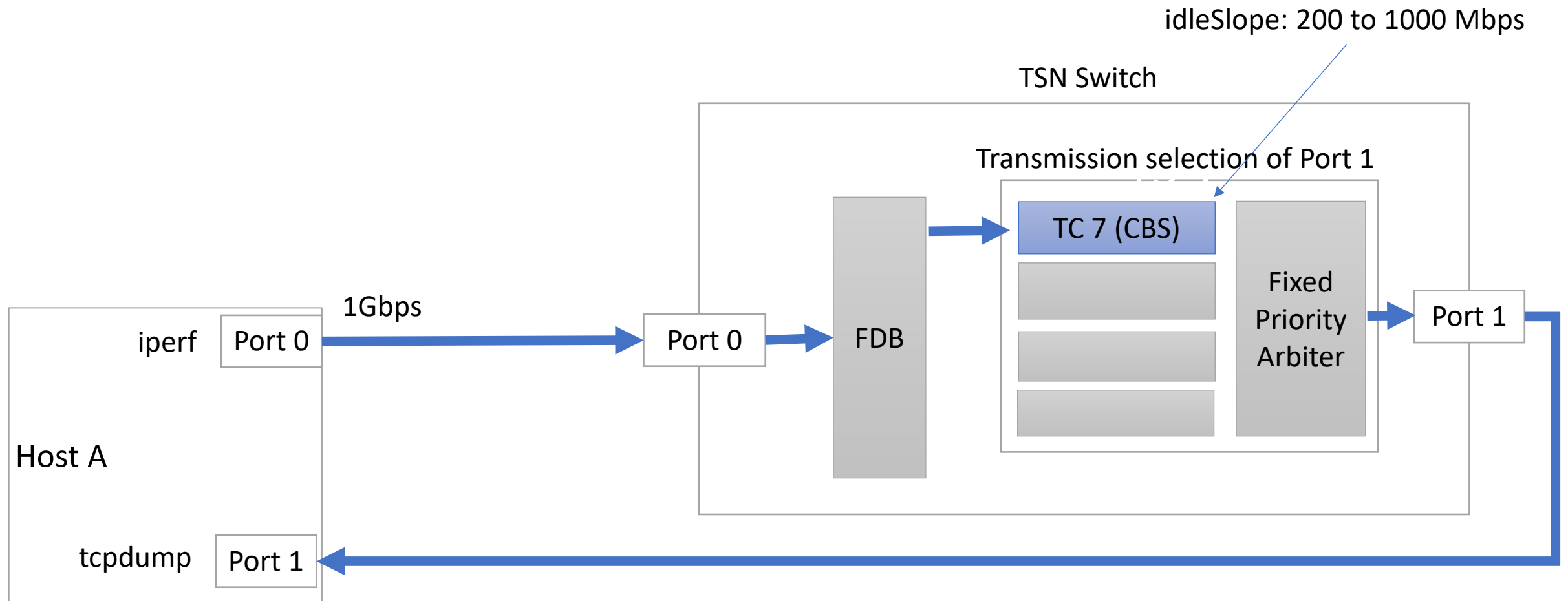
Table2: Latency in each FPGA component (1 cycle = 8 ns)

Header+ Payload (Bytes)	Eth driver (cycles)	Pre-switch FIFO (cycles)	FDB (cycles)	CBS (cycles)	PHY+TEMAC (ns)	Total (ns)
14 + 64	176	91	108	5	562	4338
14 + 1500	3048	1527	1544	5	562	61778

The latency breakdown inside FPGA was obtained by using Integrated Latency Analyzer of the FPGA development framework.

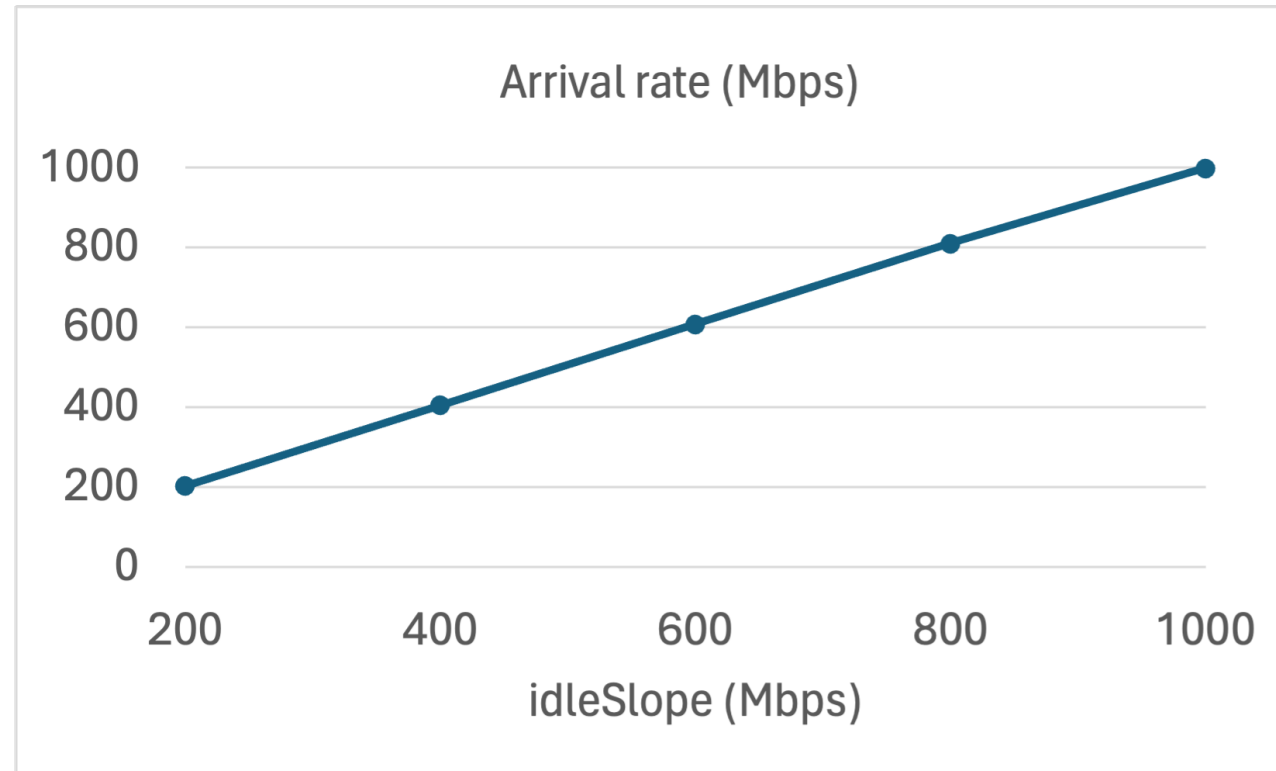
- The components internally using packet-mode FIFO cost dominant cycles.
 - The cycles consumed in Pre-switch FIFO and FDB **proportionally increase** with frame size. Those in "eth driver" proportionally increase **by 2 times** of the frame size

Rate control of CBS traffic : Setup



- We aim to observe the switch capability for rate control
- **iperf** generates UDP frames for Traffic Class 7 at the maximum frame rate of 1GbE

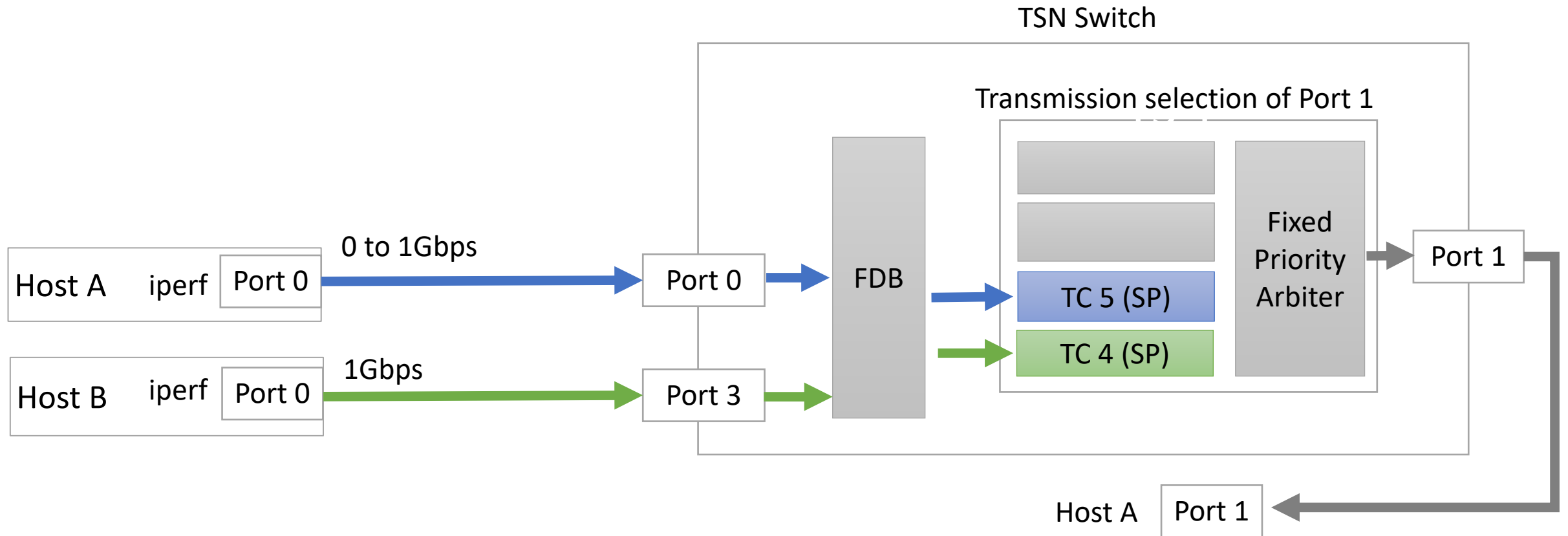
Rate control of CBS traffic: Results



Arrival rates calculated from the measured frame intervals

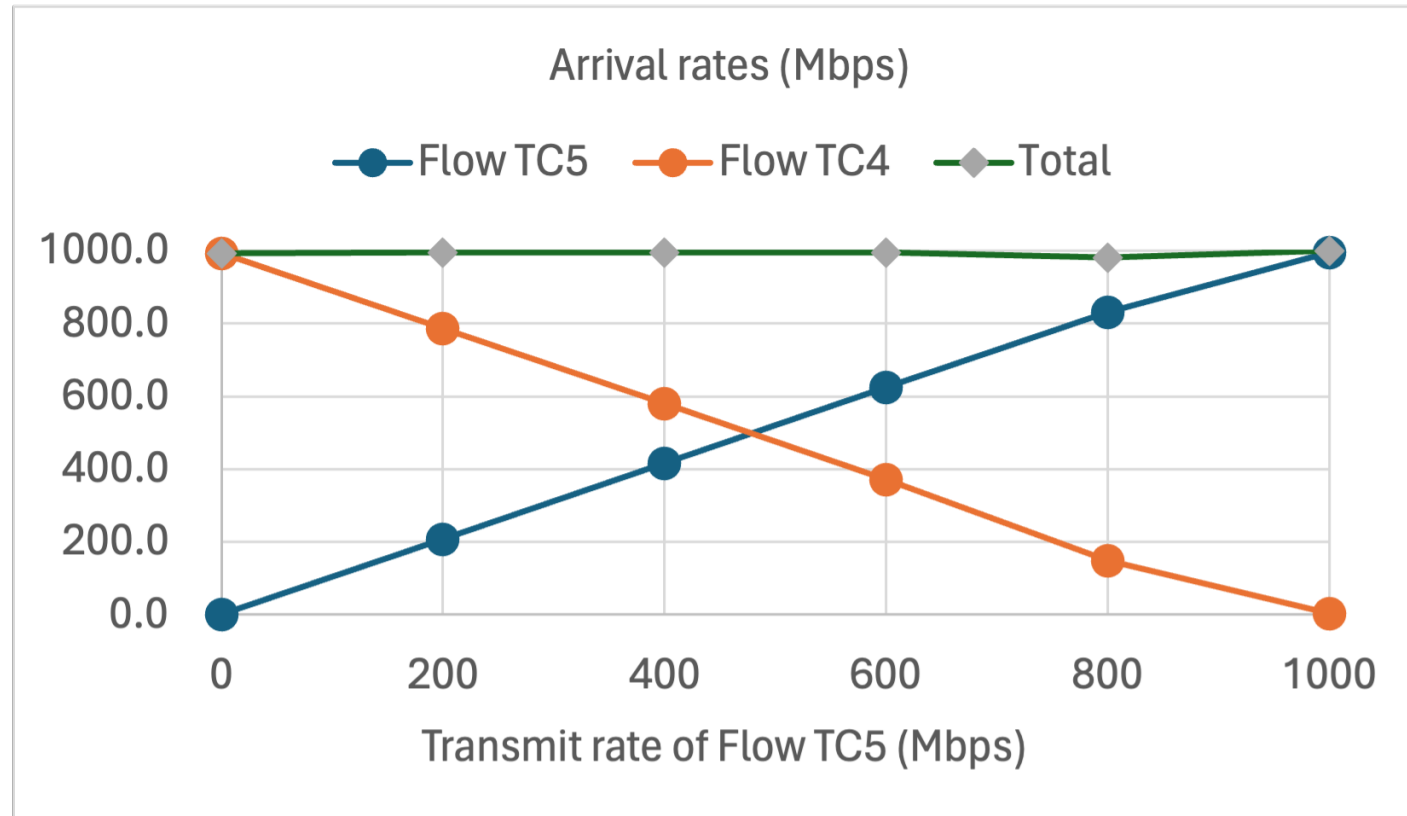
- The TSN switch **correctly delayed frames** to meet the idleSlope value
*Traffic properly shaped!

Strict Priority Control: Setup



- We aim to confirm that the fixed priority arbiter gives strict priority to a higher traffic class
- The incremental rate for TC5 and the constant 1Gbps rate for TC4

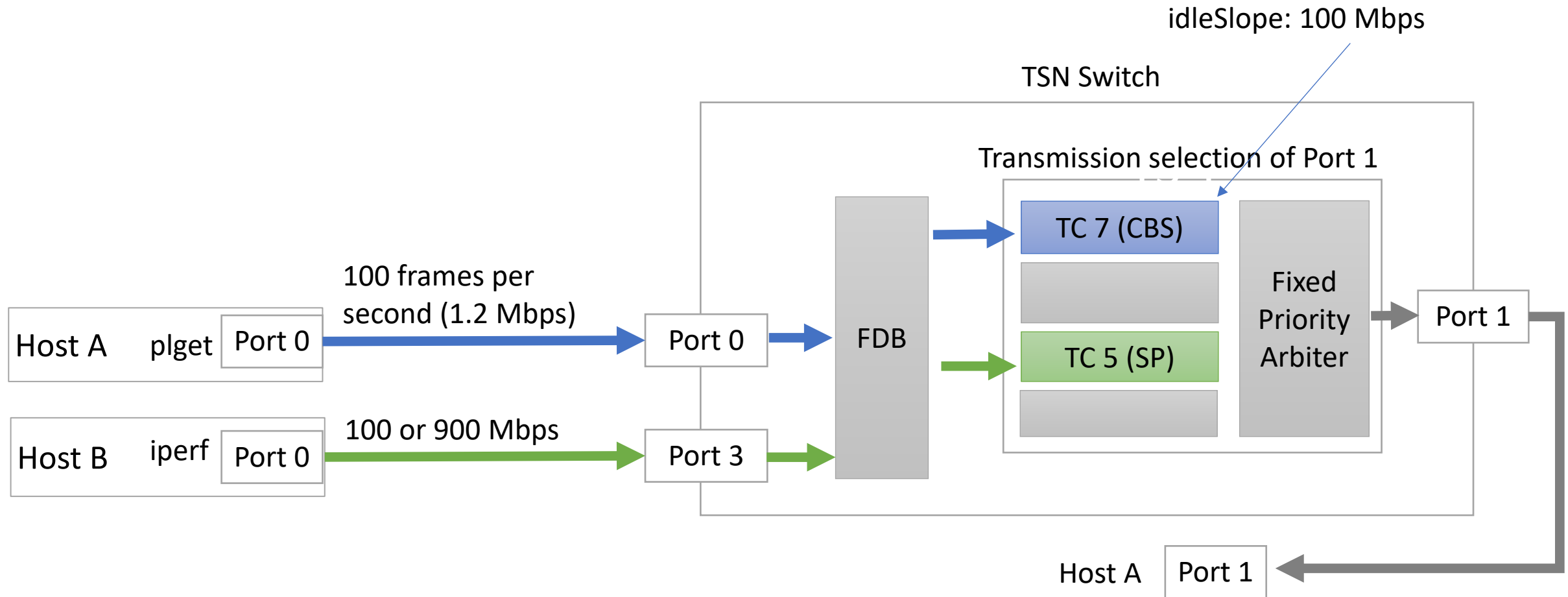
Strict Priority Control: Results



Arrival rates calculated from the measured frame intervals

- With increasing transmit rate of **TC5**, its arrival rate **kept the same** value (No dropped frames)
 - **TC4** gradually reduced its arrival rate, being **suppressed by TC5** (increasingly dropping frames)
- *Prioritization is respected!**

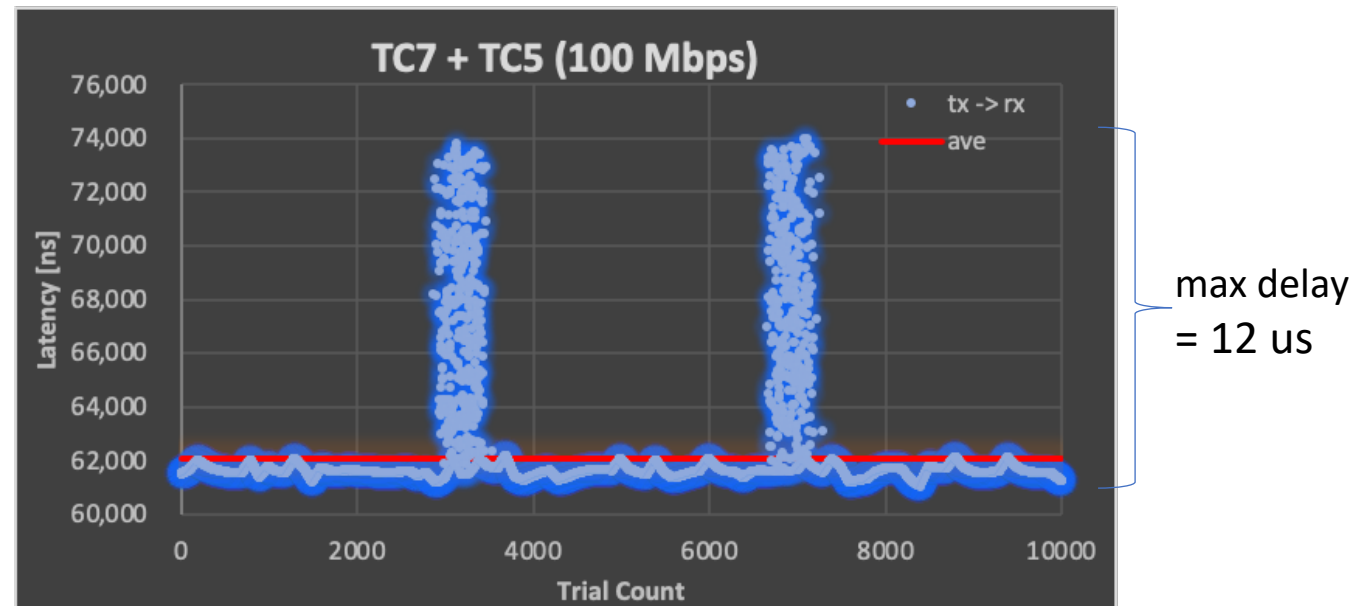
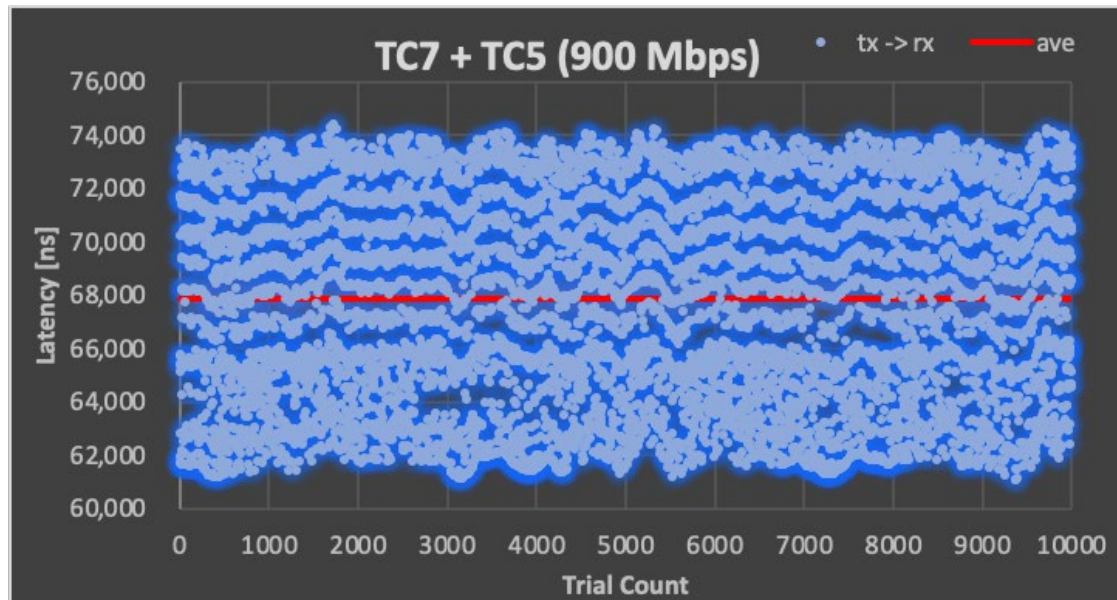
Latency bound of TC7: Setup



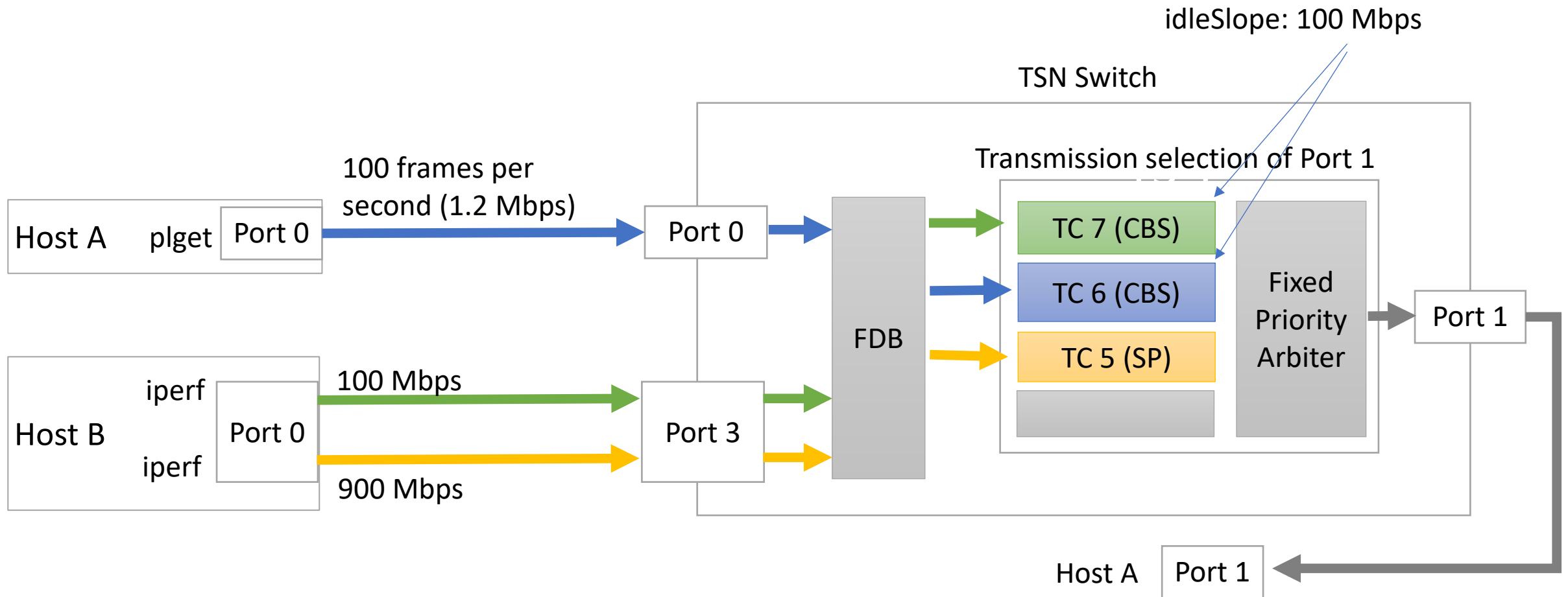
- We aim to confirm that the measured worst-case delay of the **highest** priority traffic class within the switch matches the theoretical calculated value

Latency bound of TC7 : Results

- The average latency with 900-Mbps and 100-Mbps TC5 was 68 us and 62 us.
 - ✓ When competing TC5 flow increases, high-priority TC7 frames are more likely to be delayed due to contention.
- The maximum observed TC7 latency of frames was approximately **74 us**.
 - ✓ **Increased by 12us** when compared to no contention in both case
 - ✓ 12 us is almost same as theoretical maximum transmission delay

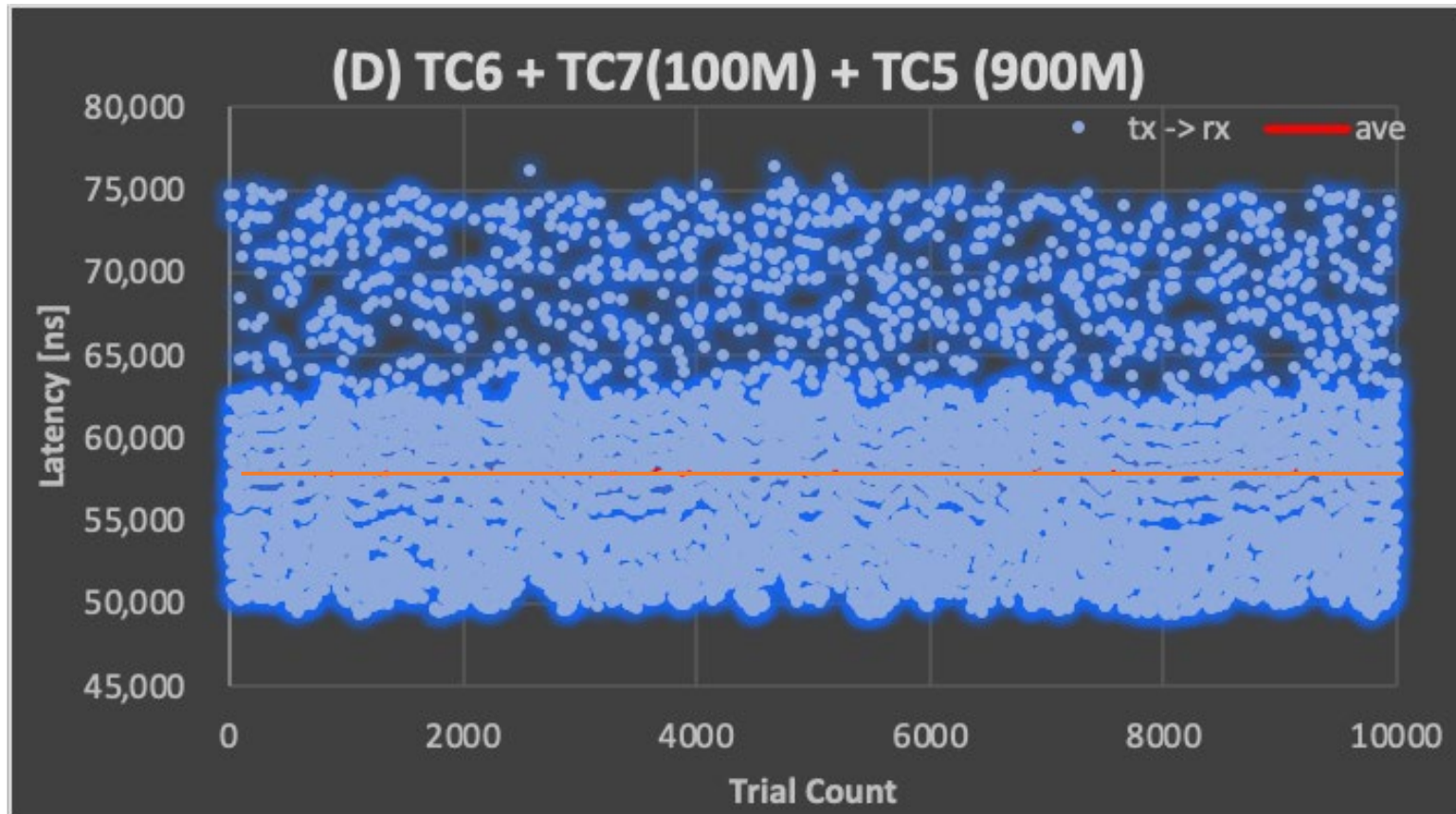


Latency bound of TC6: Setup



- We aim to confirm that the measured worst-case delay of the **second highest** priority traffic class within the switch matches the theoretical calculated value

Latency bound of TC6 (CBS): Results



25 us
≡ Twice of TC7
(Expected result)

Note that since the design was updated before this experiment, overall latencies were reduced in comparison to prior experiments.

Hardware resource utilization

Proposed switch hardware complexity

Resources / Component	MAC	Pre-switch FIFO	FDB Switch	CBS	Clock Converter	Others	Total	Utilization (%)
LUTs	9838	5536	19281	5977	266	4453	45351	22.25
FFs	15379	3716	16807	8868	448	5413	50631	12.42
LUTRAMs	1504	4096	32	4	32	230	5898	9.22
BRAMs	8	0	2	160	0	3	173	38.88

- A significant portion of the Logic Lookup Tables (LUTs) is utilized by the “FDB Switch”
 - ✓ It manages the necessary multiplexing.
- LUTRAMs and BRAM blocks are primarily consumed by the “Pre-switch FIFOs” and “CBS”.
 - ✓ They include buffers for storing processed frames
- Only **22%** of the LUTs and less than **40%** of the available BRAM blocks are utilized

Outline

- Technical Background
- Proposed Switch Architecture
- Preliminary Evaluation
- **Conclusion**

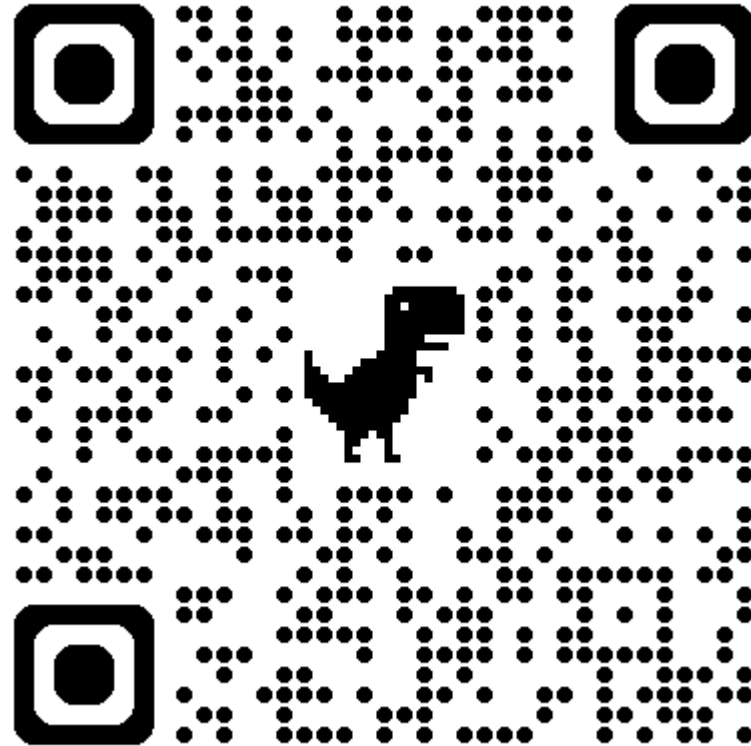
Conclusion (1/2)

- Proposed a hardware implementation of Credit Based Shaping functionalities on FPGA-based switch for Time Sensitive Networking.
- Confirmed the **proper shaping** of traffic by eliminating irregularities and efficiently forwarding streams **depending on their priority**, according to the CBS protocols rules.
- We demonstrate that our **hardware latency evaluation results** are **conform** with the CBS theoretical model.

Conclusion (2/2)

- Support Asynchronous Traffic Shaping (ATS)
 - Hardware design and Evaluation of an FPGA-based Network Switch Supporting Asynchronous Traffic Shaping for Time Sensitive Networking, IEEE Access, DOI: 10.1109/ACCESS.2024.3451448, Accepted
- All the source code including CBS and ATS is available under a very permissive open-source license (i.e., MIT License).
- For reproducible research, artifacts are also available.
- Visit <https://github.com/CCIRT/aist-tsn> .

Thank you!!



<https://github.com/CCIRT/aist-tsn>